

gstack TUTORIAL No. 2

# Build an AI-Agent End-to-End EDA System on WorkBuddy

---

A step-by-step course for postgraduate students and junior engineers

RTL → GDS on SkyWater SKY130, verified by a four-gate chain and a byte-level hash

Version 1.0 · September 2026 · Prose CC BY 4.0 · Code Apache-2.0

# What you will build — and what you must prove

---

- Build an open-source RTL → GDS flow on the SkyWater SKY130 open PDK
- Drive it with 5 gstack orchestration roles plus 2 domain AI agents
- Prove it with four verification gates and a reproducible SHA-256
- Ship it with bilingual documentation and an honest limitations section

You do not need a commercial EDA licence, an NDA, or a tape-out budget.

# Why a second tutorial?

|                 | Tutorial #1           | Tutorial #2                      |
|-----------------|-----------------------|----------------------------------|
| Carrier         | hello_world.py        | a real RTL→GDS EDA system        |
| Team            | 5 gstack review roles | 5 gstack roles × 2 domain agents |
| Proof of done   | review notes merged   | gate chain PASS + GDS SHA match  |
| Environment     | none                  | reproducible open EDA + open PDK |
| Value narrative | —                     | a system valued at ~RMB 1.25 M   |

Tutorial #1 deliberately carries no real artifact. That was the right choice — until you need one.

# The three-layer model

## Layer 1 gstack orchestration

**/plan-ceo-review → /spec → /plan-eng-review → /qa-only → /ship**  
Each role has a stop point, so decisions get audited — not absorbed.

## Layer 2 two domain agents

**AI GDS-Architect — digital: RTL→GDS, DRC, LVS, determinism**  
Senior Analog IC Architect — analog: sizing, simulation, layout intent  
Each has a handoff contract: what it receives, produces, is judged by.

## Layer 3 open EDA + PDK + gates

**Magic 8.3.681 · Netgen 1.5.323 · KLayout · ngspice · Xschem**  
SKY130A (open\_pdks 1.0.572) · Microlane P&R  
Gate 5 DRC → Gate 6 Extraction → Gate 7A LVS → Gate 7B-1R2 LVS

An agent whose output no gate examines is doing homework, not engineering.

# The five gstack roles

| Command          | Role        | Job                                    | Stops when          |
|------------------|-------------|----------------------------------------|---------------------|
| /plan-ceo-review | AI CEO      | Challenge scope, positioning, ambition | scope is defensible |
| /spec            | AI PM       | Decompose into assignable work items   | backlog is concrete |
| /plan-eng-review | AI Engineer | Lock architecture, data flow, order    | plan is buildable   |
| /qa-only         | AI QC       | Test and report — without fixing       | report is filed     |
| /ship            | AI DevOps   | Version, package, document, release    | release is tagged   |

The value is not that the AI is clever. It is that every role has a stop point.

# The two domain agents

| Agent                      | Owns                                                   | Produces                                         | Judged by                           |
|----------------------------|--------------------------------------------------------|--------------------------------------------------|-------------------------------------|
| AI GDS-Architect           | Digital backend: P&R, DRC, extraction, LVS, stream-out | GDS, extracted netlist, gate logs                | Gates 5, 6, 7A, 7B-1R2              |
| Senior Analog IC Architect | Analog front-end: topology, sizing, simulation         | Sized schematic, sim results, layout constraints | ngspice sim, DRC, (7B-2 on roadmap) |

The digital agent carries the main line. The analog agent joins at Part 9.

# The expertise gap — why five roles are not enough

| A gstack role will happily...          | ...but it does not know that                                    |
|----------------------------------------|-----------------------------------------------------------------|
| accept "LVS passed"                    | a naive LVS can be circular and prove nothing                   |
| accept "DRC clean"                     | which rule classes actually ran, and what was waived            |
| reorder build steps for tidiness       | stream order changes the GDS byte stream and breaks determinism |
| suggest "relax the check to get green" | relaxing a sign-off check is the one unforgivable move          |
| package whatever exists                | a GDS without a reproducible hash is not a release              |
| draft the positioning text             | one wrong physics number destroys the whole credibility         |

gstack roles are process experts and domain generalists by design. That is what makes them good orchestrators.

## THE TWO-KEY SYSTEM

**Process authority sits with gstack.  
Physics authority sits with the domain agents.  
On conflict, physics wins.**

---

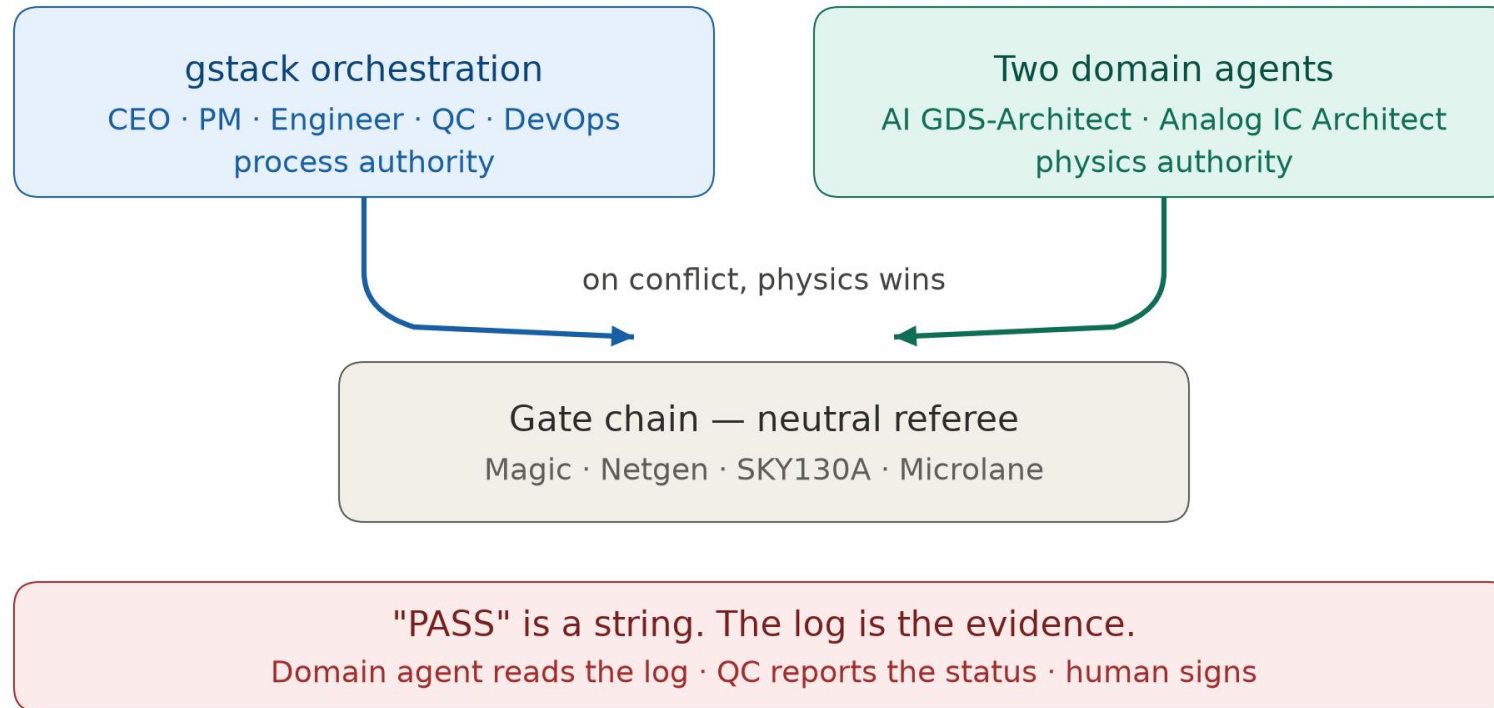
Not "5 + 2". A two-key system — 5 who decide process, 2 who decide physics.

# The authority contract — who decides what

| Decision                            | gstack      | Domain agent | Human    |
|-------------------------------------|-------------|--------------|----------|
| Scope, phasing, work breakdown      | decides     | consulted    | approves |
| PDK / stdcell library choice        | —           | DECIDES      | approves |
| What counts as a passing gate       | runs it     | DECIDES      | —        |
| Waiver of a DRC violation           | not allowed | DECIDES      | signs    |
| Interpretation of an LVS mismatch   | reports     | DECIDES      | —        |
| Release content and version         | decides     | consulted    | approves |
| Any statement with a physics number | not allowed | DECIDES      | —        |

Read the "not allowed" rows first. They are the point of the table.

# The two-key authority model — one picture



gstack holds process authority, the two domain agents hold physics authority; both feed the gate chain, and on conflict physics wins.

# Three veto rules

---

- Domain veto beats process convenience — an unsafe step does not happen, whatever the plan says
- A gate result is not a fact until the domain agent has read the log, not just the status line
- No check is ever relaxed to make a build pass — the domain agent states which is wrong, in writing

"PASS" is a string. The log is the evidence.

# The defer clause — paste into every gstack prompt

---

```
CONSULT REQUIRED. Before you finalise any statement about DRC, LVS,
extraction, determinism, or PDK rules, hand that specific claim to
the domain agent (AI GDS-Architect for digital, Senior Analog IC
Architect for analog) and quote its answer verbatim.
```

```
If the domain agent contradicts you, the domain agent wins. Do not
resolve the conflict yourself - escalate to the human with both
statements side by side.
```

One paragraph. It is the difference between sounding right and being right.

Why the agents are credible: non-circular LVS, deterministic GDS, and a 22-year chain from a Lyapunov exponent (2002) to a certified diagnostic (2023).

# Why SKY130, not a commercial PDK

---

- An NDA TSMC PDK cannot be published, redistributed or committed
- An "open-source tutorial" shipping a closed PDK is not open source — it is a leak with a README
- SKY130 is the first foundry-grade PDK under an open licence, with open DRC/LVS/RCX rule decks
- Nothing is lost: the method is process-agnostic and transfers to any PDK

This section is itself the lesson: it teaches what may be open-sourced, using a real decision.

# Course map

| Part  | What you do                                         | Time         |
|-------|-----------------------------------------------------|--------------|
| 0     | Mental model: three layers, five roles, two agents  | 30 min       |
| 1     | Install WorkBuddy, gstack and the two domain agents | 45 min       |
| 2     | Install the open EDA toolchain (three paths)        | 30 min – 4 h |
| 3     | Phase 1 — AI CEO sets the scope                     | 45 min       |
| 4     | Phase 2 — AI PM decomposes the work                 | 45 min       |
| 5     | Phase 3 — AI Engineer runs RTL → GDS                | 3–5 h        |
| 6     | Phase 4 — AI QC verifies and proves determinism     | 2–3 h        |
| 7     | Phase 5 — AI DevOps ships it                        | 1–2 h        |
| 8     | The two disciplines this tutorial actually teaches  | 30 min       |
| 9     | Bring in the analog agent                           | 2–4 h        |
| 10–12 | Troubleshooting, assessment, where to go next       | self-paced   |

Total: 10–14 h for a first pass. Every part ends with a checkpoint you must pass.

# Part 0 — Checkpoint

---

- ☐ You can name all five gstack commands and each one's stop point
- ☐ You can state, in one sentence, the difference between Tutorial #1 and #2
- ☐ You can explain why an NDA PDK cannot appear in this repository
- ☐ You can name one decision the gstack roles may NOT make, and who makes it
- ☐ You have the defer clause copied where you can paste it from

Do not advance until all five are green. The mental model is the transferable part.

# Part 1 — Install the host and the two agents

---

- 1 Download Tencent WorkBuddy, install, sign in, verify with a trivial prompt
- 2 Install the gstack skill suite; confirm @skill:gstack routes to the right skill
- 3 Create ~/.workbuddy/skills/e2e-ic-system/agents/
- 4 Save ai\_gds\_architect.md and analog\_ic\_architect.md (minimal briefs are in Part 1.3)
- 5 Restart WorkBuddy and confirm: "List the agents you can see"

Agent specs are host-agnostic — Claude Code and OpenAI Codex work the same way.

# Part 2 — Three install paths

| Path                                      | Platform            | Time             | Status              |
|-------------------------------------------|---------------------|------------------|---------------------|
| A — one-click installer (install_eda.bat) | Windows 11          | 1–3 h (compiles) | beta                |
| B — manual step-by-step                   | any                 | 2–4 h            | fully documented    |
| C — TinyTapeout VM image (recommended)    | Win / macOS / Linux | ~30 min          | upstream-maintained |

If you have never installed an open EDA toolchain, take Option C. Do not spend day one compiling.

# Pinned versions — do not drift

| Component           | Version | Commit    | Source                    |
|---------------------|---------|-----------|---------------------------|
| Magic               | 8.3.681 | 4432d7e   | RTimothyEdwards/magic     |
| Netgen              | 1.5.323 | bb8a610   | RTimothyEdwards/netgen    |
| open_pdks / SKY130A | 1.0.572 | 54435919  | RTimothyEdwards/open_pdks |
| Microlane           | —       | 87079e7f6 | htfab/microlane           |

Version drift is the single most common cause of a mysterious SHA mismatch.

# Verify before you go further

---

```
magic -dnull -noconsole --version
netgen -batch -nogui
cat $PDK_ROOT/sky130A/.config/nodeinfo.json
ls $PDK_ROOT/sky130A/libs.tech

# expected:
# node sky130A · 130 nm · open_pdks 1.0.572 (54435919)
# magic 8.3.681 (4432d7e) · 11 stdcell libs
# sky130_fd_sc_hd: 446 cells
```

If all four respond, you are ready for Phase 1.

The open PDK ships DRC, LVS and RCX rule decks — the PEX gap is in our pipeline, not the PDK.

# Phase 1 — AI CEO sets the scope

---

- Run /plan-ceo-review and paste your scope statement
- Ask explicitly: what should I cut first? Which claim is weakest?
- A good review returns four things — risk-ranked scope, a cut list, a challenge, an open question
- Write the outcome into reviews/01-ceo-review.md

If the review raises no question you cannot answer yet, the review is shallow.

# Phase 2 — Work breakdown

| ID | Work item                                   | Owner               | Judged by                |
|----|---------------------------------------------|---------------------|--------------------------|
| W1 | Toolchain install + VERSIONS.lock           | DevOps              | Checkpoint 2             |
| W2 | Agent install + handoff contract            | DevOps              | Checkpoint 1             |
| W3 | config.env + isolated launcher              | Engineer            | launcher runs            |
| W4 | Place-and-route to GDS                      | AI GDS-Architect    | Gates 5, 6               |
| W5 | Non-circular reference source export        | AI GDS-Architect    | artifact not GDS-derived |
| W6 | LVS pair (7A, 7B-1R2)                       | AI GDS-Architect    | Gates 7A, 7B-1R2         |
| W7 | Determinism proof (repeat run, compare SHA) | QC                  | SHA identical            |
| W8 | Analog sizing + simulation (optional track) | Analog IC Architect | ngspice sim              |
| W9 | Bilingual docs + release package            | DevOps              | Definition of Done       |

Every work item names the gate that judges it. No gate, no credit.

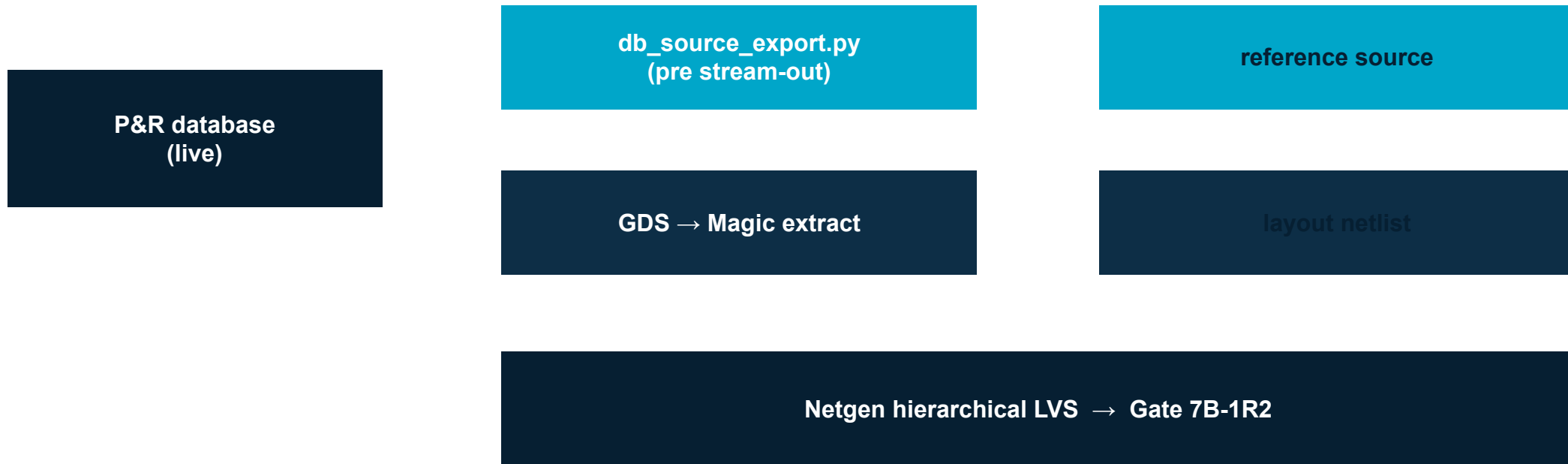
## Phase 3 — AI Engineer runs RTL → GDS

---

- Run /plan-eng-review first: lock data flow, artifact naming, gate positions
- Ask the question juniors never ask: what happens to the run directory when a gate fails?
- Dispatch the AI GDS-Architect with the four mandatory report lines
- Use the isolated launcher — never a bare shell on Windows

Mandatory report lines: artifacts with paths · gate status · GDS sha256 · what was NOT checked.

# The non-circularity rule — the intellectual core



Comparing a GDS netlist against the same GDS always passes. Two derivations from one live database do not.

Decap power/ground must come from the authoritative PDK CDL/SPICE, not from the layout.

# The four gates

| Gate                | Checks                             | Tool                 | Pass criterion                 |
|---------------------|------------------------------------|----------------------|--------------------------------|
| 5 — DRC             | geometry vs foundry rules          | Magic 8.3.681        | 0 violations (waivers listed)  |
| 6 — Extraction      | layout → netlist                   | Magic 8.3.681        | completes; warnings enumerated |
| 7A — Structural LVS | P&R netlist vs gate-level netlist  | Microlane comparator | structural match               |
| 7B-1R2 — Hier. LVS  | extracted vs P&R-DB-derived source | Netgen 1.5.323       | Circuits match uniquely.       |

A gate with no artifact did not run.

# The exit-code contract — VOID gets its own code

| Code | Meaning              | What you must do                                           |
|------|----------------------|------------------------------------------------------------|
| 0    | all gates PASS       | record the SHA and move on                                 |
| 1    | a gate FAILED        | freeze the run dir, read the log, fix the cause            |
| 2    | a gate returned VOID | the check proved nothing — re-derive the reference, re-run |
| 3    | environment error    | bad config, missing tool, or a reused RUN_DIR              |

If VOID shared a code with PASS, a reviewer skimming — or a CI job asserting only "non-zero" — could mistake it for success.

## Phase 4 — AI QC verifies, and you prove determinism

---

- Organisational rule: the person who ran a gate cannot certify it
- Run /qa-only — it reports, it does not fix
- Run the whole chain twice in clean shells and compare sha256 of the GDS
- Record everything in VERSIONS.lock; publish two baselines (Windows chain, VM chain)

Do not "fix" a SHA mismatch by relaxing the check. Investigate it.

# Phase 5 — AI DevOps ships it

## Release must contain

- VERSIONS.lock — reproducibility
- Gate logs for all four gates
- GDS + reference SHA
- README (EN) + README (ZH)
- Known limitations section
- Licence: CC BY 4.0 prose, Apache-2.0 code

## The four-file rule

- One conceptual change = 4 files × 4 checks
- Gate script · EN doc · ZH doc · review record
- Each must be re-checked
- Budget for it at the start
- Do not discover it at the end

Carried over verbatim from Tutorial #1, because it is still true.

# Discipline 1 — Lock the artifact

---

- Pinned tool versions + PYTHONHASHSEED=0 + seeded RNG + frozen GDS timestamp
- ⇒ byte-identical output ⇒ a hash you can publish
- An AI agent can produce something plausible in minutes
- The only thing between "plausible" and "engineering" is a referee that cannot be argued with

A hash is such a referee.

# Discipline 2 — Check the number, not just the status

| Case                                         | Predicted | Measured         | Error       |
|----------------------------------------------|-----------|------------------|-------------|
| Quadcopter hover power (GDA)                 | 127.8 W   | 127.4 W          | 0.29 %      |
| CTC capture critical capillary number $Ca^*$ | 0.043     | $0.04 \pm 0.006$ | inside band |

Both survive a first-principles cross-check — and both state their falsifiable assumption openly.

## THE SENTENCE THAT MATTERS

**"FM = 0.39, taken independently from propeller data — not fitted to this flight."**

---

Volunteering the assumption under which your number could be wrong is what converts a claim into evidence.

## Part 9 — Bringing in the analog agent

---

- Senior Analog IC Architect owns: schematic, sizing, simulation, corners, layout intent
- Tools: Xschem → ngspice → Magic
- Always ask it to name the one assumption most likely to be wrong
- Honest status: its gates are not yet implemented — judged by simulation and human review

22 years from a Lyapunov exponent to a certified diagnostic. The domain changed; the discipline did not.

# The workflow the analog agent enforces

| Stage               | Phases | Question it answers                                            |
|---------------------|--------|----------------------------------------------------------------|
| Specs & physics     | 1–3    | sensor model, noise / dynamic-range budget, PDK device data    |
| Architecture        | 4–6    | topology comparison, g_m/I_D first sizing, hand calculation    |
| Nominal & behaviour | 7–9    | nominal SPICE, noise contributions, behavioral sim             |
| Statistics          | 10–12  | PVT, Monte Carlo / mismatch, design centering                  |
| Layout              | 13–15  | floorplan, critical devices, progressive PEX                   |
| Post-layout         | 16–18  | noise / stability / PVT, high-sigma yield, reliability / EMIR  |
| Silicon             | 19–20  | AMS verification, tapeout, characterization, model correlation |

Learn the sequence before the tools — the sequence IS the expertise. Full 20-row table: TUTORIAL.md §9.5.

# The philosophy in one line

---

S  
p  
e  
c  
s  
  
→  
  
P  
h  
y  
s  
i  
c  
s

Every arrow is a gate: name the artifact that crosses it, or mark the phase VOID — a stage with no comparable artifact did not pass, it merely ran.

# Where AI belongs — the four levels

| Level                    | AI does                                                    | Verified by                           | Status in 2026                                                             |
|--------------------------|------------------------------------------------------------|---------------------------------------|----------------------------------------------------------------------------|
| 1 Engineering assistant  | equations, scripts, test benches, log analysis, trade-offs | the engineer reads everything         | very useful today                                                          |
| 2 Design-space optimizer | chooses W, L, I_D, C, R within your constraints            | the simulator — the only referee      | very useful, if constrained                                                |
| 3 Layout assistant       | placement, routing, constraints, parasitics                | DRC / LVS gates + layout-aware checks | fast-moving — Synopsys: AI analog layout flow, DRC-clean, fewer iterations |
| 4 Autonomous agents      | orchestrates EDA tasks end to end                          | physics-based engines, continuously   | emerging — Siemens: "self-verifying" agents; the LLM never decides alone   |

The two-key contract of this tutorial is Level 4 built from open tools: the LLM never determines correctness — the physics engine does.

# Troubleshooting — top six

| Symptom                        | Likely cause                           | Fix                                  |
|--------------------------------|----------------------------------------|--------------------------------------|
| magic: command not found       | not on PATH / Cygwin not isolated      | use the launcher; check config.env   |
| SHA differs between runs       | version drift, timestamp not frozen    | VERSIONS.lock; PYTHONHASHSEED=0      |
| SHA differs on the VM          | different PDK build and tool revisions | expected — publish a second baseline |
| Netgen setup-file error        | 6-arg form used in a sourced script    | use the 4-arg Tcl form               |
| Paths look corrupted           | MSYS path mangling                     | env -i + MSYS2_ARG_CONV_EXCL=*       |
| LVS passes suspiciously easily | circular comparison (GDS vs GDS)       | reference must come from the P&R DB  |

# Assessment — grading rubric

| Criterion                                               | Weight |
|---------------------------------------------------------|--------|
| All four gates PASS with artifacts                      | 30 %   |
| SHA reproducible and recorded                           | 20 %   |
| Limitations stated honestly and specifically            | 20 %   |
| Discipline 2 demonstrated — a number recomputed by hand | 20 %   |
| Bilingual or clearly documented output                  | 10 %   |

Labs: reproduce · break it deliberately · circular LVS · physics vs status · agent accountability.

# Where to go next

---

- PEX / RCX and Gate 7B-2 — the rule decks already ship with the PDK
- Post-layout simulation with ngspice; full STA via OpenLane/OpenSTA (needs Docker)
- Tape-out through the TinyTapeout shuttle, from \$100
- Give the analog agent a real gate chain — high effort, high value

Credits: google/skywater-pdk · open\_pdks · Magic · Netgen · Microlane · Tiny Tapeout · Zero to ASIC · SiliWiz

# What ships in the repository

## Runnable skeleton — starter/

- `run_all.sh` drives all four gates + circularity guard + fingerprint
- `run_drc.tcl` · `run_extract.tcl` · `run_lvs.tcl`
- `db_export.py` — the pre-streamout reference export
- `config.env.example` · `versions.lock.template` · `designs/`

## Audit trail — reviews/

- 01 CEO — scope, cut list, questions referred not guessed
- 02 spec — W1–W9, critical path, three escalations
- 03 eng — the circular LVS found VOID, then fixed
- 04 QA — and an eight-row "what was NOT checked" table
- 05 ship — contents, parity, licences, four-file audit

CI (`.github/workflows/selfcheck.yml`) fails if a version pin drifts between documents, or if the circularity guard disappears.

## THE ONE-LINE DIFFERENCE

**In #1 the agents review the work.  
In #2 the agents do the work —  
and are held to a verification gate.**

---

Your tutorial is complete when your four gates pass and your SHA reproduces.